Algorithms for Applications	HW0: *Getting Comfortable With Python*
CS 5112 Fall 2025	**Due: September 2, 11:59 PM ET**

Homework Instructions: The homework contains mainly three types of tasks:

- **Coding.** You can find a template for all the coding problems inside the `problems.zip` in the module. Please use these templates, to ensure the files and functions have the same names that the autograder will check for.

  Inside the `problems` directory, you will find the `tests` directory, which contains some sample test cases for the different problems. These test cases are intended to help you test and debug your solution. However, note that Gradescope will run a more comprehensive test suite. Feel free to add additional test cases to further test your code.

  In order to run the tests for a particular problem n, navigate into `problems/tests` and run: `python3 test_problem_n.py`

  If a problem requires you to submit code, it will be marked with a 💻.

- **Algorithm Description.** When a questions requires designing an algorithm, you should describe what your algorithm does in the writeup, in clear concise prose. You can cite algorithms covered in class to help your description. You should also argue for your algorithm's asymptotic run time or, in some cases when indicated in the problem, for the run time bounds of your implementation.

  If a problem requires a writeup, it will be marked with a 📝.

- **Empirical Performance Analysis.** Some questions may ask you to do an empirical performance analysis of one or more algorithms' runtime under different values of n. For these questions, you should generate at least ten test cases for the implementation, for various values of n (include both small and large instances). Then, measure the performance locally on your own system, by taking the median runtime of the implementation over ten or more iterations. Graph the resulting median run times. The x-axis should be instance size and the y-axis should be median run time. You can use any plotting library of your choice. If you have never plotted on Python before, this matplotlib turorial has some examples.

  If several subquestions in a problem ask you to provide performance analysis, please do them all in the same environment (same system and configuration), in order to have a more accurrate comparison between algorithms. You can include graphs for a same problem in the same plot, as long as the different graphs are properly labeled.

  For these types of questions, you will not need to submit the tests you generate or your code for benchmarking or plotting the algorithms. However, you should include the graphs you generate into the write-up. You should also write about what you observe from the analysis and potentially compare it to the complexity analysis. Finally, also include a description of the environment (CPU, operating system and version, amount of memory) you did the testing on.

  If a problem requires a plot, it will be marked with a 🖼️.

Submission Instructions: Hand in your solutions electronically on Gradescope. There are two active assignments for this problem set on Gradescope: one with the autograder, where you submit your code, and another one where you submit the write-up.

Coding Submission: You only need to submit the code for the specific functions we ask you to implement. While you will write additional code to generate the performance analysis graphs when required, you do not need to submit that code.

To submit to the autograder, please zip the `problems` directory we provided you with. Therefore, your zipped file should have the following structure:

```
problems
├── problem_1
│   ├── a.py
│   ├── b.py
│   └── ...
├── problem_2
│   └── a.py
└── ...
```

In order to ensure your code runs correctly, do not change the names of the files or functions we have provide you. Additionally, do not import external python libraries in these files, you don't need any libraries to solve the coding assignments. Finally, if you write any helper functions for your solutions, please include them in the same files as the functions that call them.

We have some public tests in the autograder to verify that all the required files are included, so you will be able to verify on Gradescope that your work is properly formatted shortly after submitting. We strongly encourage checking well ahead of the due date that your solutions work on the autograder, and seeking out assistance from the TAs should it not. We cannot guarantee being responsive the night the assignment is due.

Write-up Submission: Your write-up should be a nicely formatted PDF prepared using the LaTeX template on Canvas, where you can type in your answer in the `main.tex` file. If you do not have previous experience using LaTeX, we recommend using Overleaf. It is an online LaTeX editor, where you can upload and edit the template we provide. For additional advice on typesetting your work, please refer to the *resources* directory on the course's website.

Academic Integrity: You may use online sources or tools (such as code generation tools), but any tools you use should be explicitly acknowledged and you must explain how you used them. You are responsible for the correctness of submitted solutions, and we expect you to understand them and be able to explain them when asked by teaching staff.

Collaboration Policy: Collaboration (in groups of up to three students) is encouraged while solving the problems, but:

1. list the netids of those in your group;
2. you may discuss ideas and approaches, but you should not write a detailed argument or code outline together;
3. notes of your discussions should be limited to drawings and a few keywords; you must

write up the solutions and write code on your own.

Welcome to CS 5112, Algorithms and Data Structures for Applications!

In this mini-assignment, you are expected to gain an understanding of the basic concepts of the Python programming language, as well as gain some familiarity with recursion and dynamic programming.

Students are expected to solve these problems by browsing the internet to find relevant documentation to enable them to solve the problems. **We strongly advise against** using AI tools to solve these basic problems.

Problem 1.

In this problem, you will practice basic Python control flow and collection operations. Complete the functions in `problem_1/a.py`, `problem_1/b.py`, `problem_1/c.py`, `problem_1/d.py`, and `problem_1/e.py`.

(a) (*1 point*) (💻)

Implement `categorize_number` in `problem_1/a.py` using if-elif-else statements. The function should return `"negative"`, `"zero"`, or `"positive"` based on the input number.

(b) (*2 points*) (💻)

Implement the list functions `find_index` and `reverse_list` in `problem_1/b.py`.

`find_index` should return the index of a value in a list, or -1 if the value is not present. Do not use the built-in `.index()` method.

`reverse_list` should return a reversed copy of a list without using the built-in `reverse` method.

(c) (*2 points*) (💻)

Implement `process_numbers` using a for loop with `break` and `continue`. Put your implementation in `problem_1/c.py`.

The function should print each positive number in order. It should skip zeros using `continue`, and it should stop processing at the first negative number using `break`.

(d) (*1 point*) (💻)

Implement `string_lengths`. Given a list of strings, return a dictionary whose keys are the strings and whose values are the lengths of those strings. Put your implementation in `problem_1/d.py`.

(e) (*4 points*) (💻)

Implement the conditional list-comprehension functions in `problem_1/e.py`.

1. `even_values_keys`: given a dictionary, return the keys whose values are even.
2. `square_evens`: given a list of numbers, return the squares of the even numbers.
3. `long_strings`: given a list of strings, return the strings with length greater than 5.
4. `convert_to_celsius`: convert Fahrenheit temperatures to Celsius strings, appending `Freezing` for temperatures at or below 32 degrees Fahrenheit.

Submit your completed `problem_1/a.py`, `problem_1/b.py`, `problem_1/c.py`, `problem_1/d.py`, and `problem_1/e.py` files.

Problem 2.

In this problem, you will practice mutating Python lists. Complete the functions in `problem_2/a.py`, `problem_2/b.py`, `problem_2/c.py`, and `problem_2/d.py`.

(a) (*1 point*) (💻)

Implement `add_element` in `problem_2/a.py`. The function should append the given element to the end of the input list.

(b) (*1 point*) (💻)

Implement `insert_element_at` in `problem_2/b.py`. The function should insert the given element at the specified index of the input list.

(c) (*1 point*) (💻)

Implement `remove_element` in `problem_2/c.py`. The function should remove the first occurrence of the element from the input list. If the element is not present, the list should remain unchanged.

(d) (*1 point*) (💻)

Implement `pop_element_at` in `problem_2/d.py`. The function should pop and return the element at the specified index. If the index is invalid, return `None` and leave the list unchanged.

Submit your completed `problem_2/a.py`, `problem_2/b.py`, `problem_2/c.py`, and `problem_2/d.py` files.

Problem 3.

In this problem, you will practice reading data from Python dictionaries. Complete the functions in `problem_3/a.py`.

(a) (*4 points*) (💻)

Implement the dictionary helper functions in `problem_3/a.py`.

1. `get_keys`: return a list of all keys in the dictionary.
2. `get_values`: return a list of all values in the dictionary.
3. `get_items`: return a list of all key-value pairs as tuples.
4. `get_value`: return the value for the given key if it exists, otherwise return the default value.

Submit your completed `problem_3/a.py` file.

Problem 4.

In this problem, you will practice list comprehensions and functional-style Python tools. Complete the functions in `problem_4/a.py` and `problem_4/b.py`.

(a) (*2 points*) (💻)

Implement the list-comprehension functions in `problem_4/a.py`.

1. `square_all`: return a list containing the square of every number in the input list.
2. `filter_even`: return a list containing only the even numbers from the input list.

(b) (*1 point*) (💻)

Implement `product_of_all` in `problem_4/b.py`. The function should use `functools.reduce` and a lambda function to return the product of all numbers in the input list.

Submit your completed `problem_4/a.py` and `problem_4/b.py` files.

Problem 5.

In this problem, you will implement factorial in two different ways. Complete the functions in `problem_5/a.py` and `problem_5/b.py`, and briefly describe each approach in your write-up.

(a) (*2 points*) (📝💻)

Implement `factorial_reduce` in `problem_5/a.py`. The function should use `functools.reduce` and a lambda function to compute $n!$. Include a brief write-up describing your solution.

(b) (*2 points*) (📝💻)

Implement `factorial_rec` in `problem_5/b.py`. The function should use recursion to compute $n!$. Include a brief write-up describing your solution.

Submit your completed `problem_5/a.py` and `problem_5/b.py` files.